Modul Metode Numerik

Metode Pencarian Akar Persamaan Nonlinear (1-3)

Dr. Indira Puteri Kinasih

Daftar Isi

Pendahuluan 1
Tujuan Praktikum 1

Praktikum 1 — Metode *Bisection* **2**
Ide Dasar 2
Perhitungan Manual 2
Iterasi 1 2
Iterasi 2 3
Iterasi 3 3
Latihan Perhitungan Manual 3
Algoritma *Bisection* 3
Implementasi R 4
Eksekusi Program 4
Pertanyaan 5

Praktikum 2 — Metode *Regula Falsi* **6**
Ide Dasar 6
Perhitungan Manual 6
Algoritma 7
Implementasi R 7
Pertanyaan 8

Praktikum 3 — Metode *fixed point* **9**
Ide Dasar 9
Contoh 9
Perhitungan Manual 10
Konvergensi *fixed point* 10
Implementasi R 10
Latihan 12

Praktikum 4 — Metode Newton-Raphson **13**
Ide Dasar 13
Perhitungan Manual 13
Implementasi R 14
Pertanyaan 15

Praktikum 5 — Metode *Secant* **16**
Ide Dasar 16
Perhitungan Manual 16
Implementasi R 17

Visualisasi dan Perbandingan **19**

Praktikum 6 — Menerapkan Metode pada Berbagai Fungsi **21**
Tugas 1 — Penerapan Terbimbing 21
Tugas 2 — Mengubah Fungsi 22
Tugas 3 — Eksperimen tebakan awal 22
Tugas 4 — Eksperimen Nilai Toleransi 23
Tugas 5 — Perbandingan Metode 23
Tugas 6 — Analisis Konvergensi 24

Refleksi **25**

Tugas Akhir Praktikum **26**

Ringkasan Lima Metode **27**

Pendahuluan

Praktikum dilakukan menggunakan R sebagai alat bantu komputasi dan mengacu pada Modul 1 yang telah dibagikan di pertemuan sebelumnya. Pastikan R dan RStudio telah ter*install* sebelum praktikum dimulai. Sebagian besar praktikum hanya menggunakan fungsi dasar R.

Untuk membuat grafik, digunakan `ggplot2`.

```
library(ggplot2)
```

Namun, penggunaan R bukan tujuan utama praktikum. Hal yang lebih penting adalah memahami bagaimana setiap metode bekerja.

Oleh karena itu, setiap metode akan dipelajari dengan urutan:

> **Ide dasar → Perhitungan manual → Algoritma → Implementasi R → Penerapan → Eksperimen → Analisis**

Tujuan Praktikum

Setelah mengikuti praktikum ini, mahasiswa diharapkan mampu:

1. Menjelaskan konsep dasar pencarian akar persamaan nonlinear.
2. Menjelaskan ide dasar lima metode pencarian akar.
3. Melakukan beberapa iterasi secara manual.
4. Menuliskan langkah algoritmik suatu metode.
5. Mengimplementasikan metode menggunakan R.
6. Mengubah fungsi dan parameter pada kode R.
7. Menentukan akar persamaan nonlinear secara numerik.
8. Membandingkan kecepatan dan karakteristik beberapa metode.
9. Menjelaskan kondisi ketika suatu metode dapat mengalami kegagalan atau tidak konvergen.
10. Menginterpretasikan hasil komputasi numerik.

Praktikum 1 — Metode *Bisection*

Ide Dasar

Ide utama metode *Bisection* adalah:

> **mencari akar dengan cara mempersempit/mengurung interval yang mengandung akar.**

Jika pada suatu interval $[a, b]$ berlaku

$$f(a)f(b) < 0,$$

maka terdapat setidaknya satu akar di antara a dan b, dengan asumsi f kontinu. Titik tengah interval dihitung dengan

$$c = \frac{a+b}{2}.$$

Kemudian diperiksa tanda $f(c)$. Jika $f(a)f(c) < 0$, maka akar berada pada interval $[a, c]$. Jika $f(c)f(b) < 0$, maka akar berada pada interval $[c, b]$. Proses tersebut diulang sampai diperoleh ketelitian yang diinginkan.

Perhitungan Manual

Gunakan $f(x) = x^3 - x - 2$ dengan interval awal $[a, b] = [1, 2]$.

Iterasi 1

Titik tengah:

$$c_1 = \frac{1+2}{2} = 1.5.$$

Hitung:

$$f(1.5) = 1.5^3 - 1.5 - 2 = -0.125.$$

Karena $f(1) < 0$ dan $f(1.5) < 0$, akar belum berada pada interval $[1, 1.5]$.

Karena $f(1.5) < 0$ dan $f(2) > 0$, diperoleh interval baru adalah

$$[1.5, 2].$$

Iterasi 2

$$c_2 = \frac{1.5 + 2}{2} = 1.75.$$

Hitung:

$$f(1.75) = 1.75^3 - 1.75 - 2 = 1.609375.$$

Karena $f(1.5) < 0$ dan $f(1.75) > 0$, diperoleh interval baru adalah

$$[1.5, 1.75].$$

Iterasi 3

$$c_3 = \frac{1.5 + 1.75}{2} = 1.625.$$

Hitung:

$$f(1.625) = 1.625^3 - 1.625 - 2.$$

Lanjutkan perhitungan!

Latihan Perhitungan Manual

Lengkapi minimal 5 iterasi berikut.

Iterasi	a	b	c	$f(c)$	Interval Baru
1	1	2	1.5	-0.125	[1.5, 2]
2	1.5	2	1.75	1.609375	[1.5, 1.75]
3					
4					
5					

Algoritma *Bisection*

Secara umum:

1. Tentukan a dan b.
2. Pastikan $f(a)f(b) < 0$.
3. Hitung titik tengah

$$c = \frac{a + b}{2}.$$

4. Hitung (f(c)).
5. Tentukan interval baru berdasarkan perubahan tanda.
6. Ulangi proses.
7. Berhenti ketika interval atau error telah memenuhi toleransi.

Implementasi R

```r
bisection <- function(f, a, b, tol = 1e-6, max_iter = 100) {

  if (f(a) * f(b) > 0) {
    stop("Interval tidak mengandung perubahan tanda.")
  }

  hasil <- data.frame()

  for (i in 1:max_iter) {

    c <- (a + b) / 2
    fc <- f(c)

    hasil <- rbind(
      hasil,
      data.frame(
        iterasi = i,
        a = a,
        b = b,
        c = c,
        f_c = fc,
        error = abs(b - a)
      )
    )

    if (abs(b - a) < tol || abs(fc) < tol) {
      break
    }

    if (f(a) * fc < 0) {
      b <- c
    } else {
      a <- c
    }
  }

  list(
    akar = c,
    iterasi = hasil
  )
}
```

Eksekusi Program

Definisikan fungsi:

```r
f <- function(x) {
  x^3 - x - 2
}
```

Jalankan:

```r
hasil_bisection <- bisection(
  f = f,
  a = 1,
  b = 2
)

hasil_bisection$akar
```

```
[1] 1.52138
```

Tampilkan tabel iterasi:

```r
hasil_bisection$iterasi
```

	iterasi	a	b	c	f_c	error
1	1	1.000000	2.000000	1.500000	-1.250000e-01	1.000000e+00
2	2	1.500000	2.000000	1.750000	1.609375e+00	5.000000e-01
3	3	1.500000	1.750000	1.625000	6.660156e-01	2.500000e-01
4	4	1.500000	1.625000	1.562500	2.521973e-01	1.250000e-01
5	5	1.500000	1.562500	1.531250	5.911255e-02	6.250000e-02
6	6	1.500000	1.531250	1.515625	-3.405380e-02	3.125000e-02
7	7	1.515625	1.531250	1.523438	1.225042e-02	1.562500e-02
8	8	1.515625	1.523438	1.519531	-1.097125e-02	7.812500e-03
9	9	1.519531	1.523438	1.521484	6.221756e-04	3.906250e-03
10	10	1.519531	1.521484	1.520508	-5.178886e-03	1.953125e-03
11	11	1.520508	1.521484	1.520996	-2.279443e-03	9.765625e-04
12	12	1.520996	1.521484	1.521240	-8.289059e-04	4.882812e-04
13	13	1.521240	1.521484	1.521362	-1.034331e-04	2.441406e-04
14	14	1.521362	1.521484	1.521423	2.593543e-04	1.220703e-04
15	15	1.521362	1.521423	1.521393	7.795631e-05	6.103516e-05
16	16	1.521362	1.521393	1.521378	-1.273947e-05	3.051758e-05
17	17	1.521378	1.521393	1.521385	3.260816e-05	1.525879e-05
18	18	1.521378	1.521385	1.521381	9.934278e-06	7.629395e-06
19	19	1.521378	1.521381	1.521379	-1.402611e-06	3.814697e-06
20	20	1.521379	1.521381	1.521380	4.265829e-06	1.907349e-06
21	21	1.521379	1.521380	1.521380	1.431608e-06	9.536743e-07

Pertanyaan

1. Apakah hasil R sama dengan hasil perhitungan manual?
2. Berapa iterasi yang diperlukan?
3. Berapa nilai akar yang diperoleh?
4. Apa yang terjadi pada panjang interval dari satu iterasi ke iterasi berikutnya?
5. Mengapa *Bisection* disebut sebagai metode yang stabil?

Praktikum 2 — Metode *Regula Falsi*

Ide Dasar

Regula Falsi memiliki prinsip yang mirip dengan Bisection, yaitu mempertahankan interval yang memiliki perubahan tanda. Perbedaannya adalah titik baru tidak diperoleh dari titik tengah. Regula Falsi menggunakan **garis lurus yang menghubungkan dua titik** pada grafik fungsi.

Jika terdapat dua titik

$$(a, f(a))$$

dan

$$(b, f(b)),$$

maka titik potong garis tersebut dengan sumbu-x diberikan oleh

$$c = \frac{af(b) - bf(a)}{f(b) - f(a)}.$$

Jadi ide dasarnya adalah:

> **menggunakan garis lurus untuk memperkirakan lokasi akar.**

Perhitungan Manual

Gunakan $f(x) = x^3 - x - 2$ dengan $a = 1, \qquad b = 2.$

Diketahui: $f(1) = -2$ dan $f(2) = 4$. Maka:

$$c = \frac{1(4) - 2(-2)}{4 - (-2)}$$

sehingga $c = \frac{8}{6} = 1.3333$.

Hitung:

$$f(1.3333) \approx -0.9629.$$

Karena $f(c) < 0$, sedangkan $f(b) > 0$, interval baru menjadi

$$[1.3333, 2].$$

Lanjutkan perhitungan hingga minimal 4 iterasi.

Algoritma

1. Tentukan a dan b.
2. Pastikan $f(a)f(b) < 0$.
3. Hitung

$$c = \frac{af(b) - bf(a)}{f(b) - f(a)}.$$

4. Hitung $f(c)$.
5. Perbarui interval berdasarkan perubahan tanda.
6. Ulangi.
7. Berhenti ketika kriteria toleransi terpenuhi.

Implementasi R

```r
regula_falsi <- function(f, a, b, tol = 1e-6, max_iter = 100) {

  if (f(a) * f(b) > 0) {
    stop("Interval tidak mengandung perubahan tanda.")
  }

  hasil <- data.frame()

  for (i in 1:max_iter) {

    c <- (a * f(b) - b * f(a)) / (f(b) - f(a))
    fc <- f(c)

    hasil <- rbind(
      hasil,
      data.frame(
        iterasi = i,
        a = a,
        b = b,
        c = c,
        f_c = fc
      )
    )

    if (abs(fc) < tol) {
      break
    }

    if (f(a) * fc < 0) {
      b <- c
    } else {
      a <- c
    }
  }

```

```
  list(
    akar = c,
    iterasi = hasil
  )
}
```

Jalankan:

```
hasil_rf <- regula_falsi(f, 1, 2)

hasil_rf$akar
```

```
[1] 1.52138
```

```
hasil_rf$iterasi
```

```
   iterasi        a b        c           f_c
1        1 1.000000 2 1.333333 -9.629630e-01
2        2 1.333333 2 1.462687 -3.333389e-01
3        3 1.462687 2 1.504019 -1.018180e-01
4        4 1.504019 2 1.516331 -2.989480e-02
5        5 1.516331 2 1.519919 -8.675066e-03
6        6 1.519919 2 1.520957 -2.508805e-03
7        7 1.520957 2 1.521258 -7.248228e-04
8        8 1.521258 2 1.521344 -2.093499e-04
9        9 1.521344 2 1.521370 -6.046133e-05
10      10 1.521370 2 1.521377 -1.746113e-05
11      11 1.521377 2 1.521379 -5.042711e-06
12      12 1.521379 2 1.521379 -1.456314e-06
13      13 1.521379 2 1.521380 -4.205770e-07
```

Pertanyaan

1. Apa perbedaan utama *Bisection* dan Regula Falsi?
2. Mengapa Regula Falsi tidak menggunakan titik tengah?
3. Bandingkan jumlah iterasi *Regula Falsi* dan *Bisection*.
4. Apakah Regula Falsi selalu lebih cepat?

Praktikum 3 — Metode *fixed point*

Ide Dasar

Metode *fixed point* mengubah persamaan

$$f(x) = 0$$

menjadi bentuk

$$x = g(x).$$

Kemudian digunakan proses iterasi:

$$x_{n+1} = g(x_n).$$

Ide dasarnya:

> **gunakan nilai sebelumnya untuk menghasilkan nilai berikutnya.**

Berbeda dengan Bisection dan Regula Falsi, metode *fixed point* tidak secara langsung mempertahankan interval yang mengandung perubahan tanda.

Contoh

Gunakan persamaan

$$x^2 + x - 2 = 0.$$

Persamaan tersebut dapat ditulis menjadi

$$x^2 = 2 - x.$$

Sehingga

$$x = \sqrt{2 - x}.$$

Dengan demikian:

$$g(x) = \sqrt{2 - x}.$$

Gunakan tebakan awal:

$$x_0 = 0.$$

Perhitungan Manual

Iterasi pertama:

$$x_1 = g(x_0)$$

$$x_1 = \sqrt{2 - 0} = \sqrt{2} = 1.4142.$$

Iterasi kedua:

$$x_2 = \sqrt{2 - 1.4142}$$

$$x_2 \approx 0.7654.$$

Iterasi berikutnya:

$$x_3 = \sqrt{2 - 0.7654}.$$

Mahasiswa melanjutkan sampai minimal 5 iterasi.

Iterasi	x_n	$g(x_n)$	$\lvert x_{n+1} - x_n \rvert$
0	0	1.4142	1.4142
1	1.4142	0.7654	0.6488
2	0.7654	...	...
3	...	...	...
4	...	...	...

Konvergensi *fixed point*

Tidak semua bentuk

$$x = g(x)$$

akan menghasilkan metode yang konvergen. Salah satu kondisi yang umum digunakan adalah

$$|g'(x^*)| < 1$$

di sekitar *fixed point* x^*.

Hal ini menunjukkan bahwa pemilihan bentuk $g(x)$ sangat penting.

Implementasi R

```r
fixed_point <- function(g, x0, tol = 1e-6, max_iter = 100) {

  hasil <- data.frame()

  x <- x0

  for (i in 1:max_iter) {

    x_new <- g(x)
    error <- abs(x_new - x)

    hasil <- rbind(
      hasil,
      data.frame(
        iterasi = i,
        x_lama = x,
        x_baru = x_new,
        error = error
      )
    )

    if (error < tol) {
      x <- x_new
      break
    }

    x <- x_new
  }

  list(
    akar = x,
    iterasi = hasil
  )
}
```

Jalankan:

```r
g <- function(x) {
  sqrt(2 - x)
}

hasil_fp <- fixed_point(
  g = g,
  x0 = 0
)

hasil_fp$akar
```

```
[1] 0.9999998
```

```r
hasil_fp$iterasi
```

```
  iterasi    x_lama    x_baru        error
1       1 0.0000000 1.4142136 1.414214e+00
2       2 1.4142136 0.7653669 6.488467e-01
```

```
3         3 0.7653669 1.1111405 3.457736e-01
4         4 1.1111405 0.9427935 1.683470e-01
5         5 0.9427935 1.0282055 8.541201e-02
6         6 1.0282055 0.9857964 4.240910e-02
7         7 0.9857964 1.0070768 2.128038e-02
8         8 1.0070768 0.9964553 1.062143e-02
9         9 0.9964553 1.0017708 5.315431e-03
10       10 1.0017708 0.9991142 2.656540e-03
11       11 0.9991142 1.0004428 1.328564e-03
12       12 1.0004428 0.9997786 6.642086e-04
13       13 0.9997786 1.0001107 3.321227e-04
14       14 1.0001107 0.9999446 1.660568e-04
15       15 0.9999446 1.0000277 8.302953e-05
16       16 1.0000277 0.9999862 4.151448e-05
17       17 0.9999862 1.0000069 2.075731e-05
18       18 1.0000069 0.9999965 1.037864e-05
19       19 0.9999965 1.0000017 5.189323e-06
20       20 1.0000017 0.9999991 2.594660e-06
21       21 0.9999991 1.0000004 1.297330e-06
22       22 1.0000004 0.9999998 6.486652e-07
```

Latihan

Gunakan persamaan

$$x^3 + x - 1 = 0.$$

Ubah persamaan tersebut menjadi bentuk

$$x = g(x).$$

Kemudian:

1. Tentukan $g(x)$.
2. Tentukan tebakan awal.
3. Lakukan 5 iterasi manual.
4. Implementasikan menggunakan R.
5. Bandingkan hasil manual dan R.
6. Periksa apakah metode konvergen.

Praktikum 4 — Metode Newton-Raphson

Ide Dasar

Newton-Raphson menggunakan **garis singgung** untuk memperkirakan lokasi akar. Jika kita berada pada titik (x_n), garis singgung grafik (f(x)) digunakan untuk memperkirakan titik potong dengan sumbu-(x).

Rumus iterasinya adalah

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

Ide dasarnya:

> **bergerak dari satu titik ke titik berikutnya menggunakan kemiringan garis singgung.**

Perhitungan Manual

Gunakan

$$f(x) = x^3 - x - 2.$$

Turunannya:

$$f'(x) = 3x^2 - 1.$$

Gunakan

$$x_0 = 1.5.$$

Maka:

$$f(1.5) = -0.125$$

dan

$$f'(1.5) = 3(1.5)^2 - 1 = 5.75.$$

Sehingga

$$x_1 = 1.5 - \frac{-0.125}{5.75}.$$

Maka:

$$x_1 \approx 1.52174.$$

Lakukan beberapa iterasi berikutnya secara manual.

Iterasi	x_n	$f(x_n)$	$f'(x_n)$	x_{n+1}
0	1.50000	-0.12500	5.75000	1.52174
1	1.52174	...	...	...
2	...	...	...	...
3	...	...	...	...

Implementasi R

```r
newraph <- function(f, df, x0, tol = 1e-6, max_iter = 100) {

  hasil <- data.frame()

  x <- x0

  for (i in 1:max_iter) {

    fx <- f(x)
    dfx <- df(x)

    if (dfx == 0) {
      stop("Turunan bernilai nol.")
    }

    x_new <- x - fx / dfx
    error <- abs(x_new - x)

    hasil <- rbind(
      hasil,
      data.frame(
        iterasi = i,
        x_lama = x,
        f_x = fx,
        df_x = dfx,
        x_baru = x_new,
        error = error
      )
    )

    if (error < tol) {
      x <- x_new
      break
    }
```

```
    x <- x_new
  }

  list(
    akar = x,
    iterasi = hasil
  )
}
```

Definisikan fungsi dan turunannya:

```
f <- function(x) {
  x^3 - x - 2
}

df <- function(x) {
  3*x^2 - 1
}
```

Jalankan:

```
hasil_newraph <- newraph(
  f = f,
  df = df,
  x0 = 1.5
)

hasil_newraph$akar
```

```
[1] 1.52138
```

```
hasil_newraph$iterasi
```

```
  iterasi   x_lama           f_x    df_x   x_baru        error
1       1 1.500000 -1.250000e-01 5.75000 1.521739 2.173913e-02
2       2 1.521739  2.136928e-03 5.94707 1.521380 3.593245e-04
3       3 1.521380  5.893874e-07 5.94379 1.521380 9.916021e-08
```

Pertanyaan

1. Mengapa Newton-Raphson membutuhkan turunan?
2. Apa yang terjadi jika (f'(x_n)=0)?
3. Mengapa Newton-Raphson dapat mencapai akar dengan sedikit iterasi?
4. Apakah Newton-Raphson selalu konvergen?
5. Bagaimana pengaruh pemilihan (x_0)?

Praktikum 5 — Metode *Secant*

Ide Dasar

Metode *Secant* memiliki ide yang mirip dengan Newton-Raphson. Perbedaannya:

> Newton-Raphson menggunakan turunan, sedangkan *Secant* memperkirakan kemiringan menggunakan dua titik.

Kemiringan garis *Secant* antara x_{n-1} dan x_n digunakan untuk menentukan titik berikutnya. Rumusnya adalah:

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.$$

Perhitungan Manual

Gunakan

$$f(x) = x^3 - x - 2.$$

Gunakan dua tebakan awal:

$$x_0 = 1, \qquad x_1 = 2.$$

Maka:

$$f(x_0) = -2$$

dan

$$f(x_1) = 4.$$

Hitung:

$$x_2 = 2 - 4\frac{2-1}{4-(-2)}.$$

Sehingga:

$$x_2 = 2 - \frac{4}{6} = 1.3333.$$

Lanjutkan perhitungan secara manual.

Iterasi	x_{n-1}	x_n	$f(x_{n-1})$	$f(x_n)$	x_{n+1}
1	1	2	-2	4	1.3333
2	2	1.3333	4	-0.9629	...
3					
4					

Implementasi R

```r
secant <- function(f, x0, x1, tol = 1e-6, max_iter = 100) {

  hasil <- data.frame()

  for (i in 1:max_iter) {

    f0 <- f(x0)
    f1 <- f(x1)

    if (f1 - f0 == 0) {
      stop("Penyebut bernilai nol.")
    }

    x2 <- x1 - f1 * (x1 - x0) / (f1 - f0)

    error <- abs(x2 - x1)

    hasil <- rbind(
      hasil,
      data.frame(
        iterasi = i,
        x_sebelumnya = x0,
        x_sekarang = x1,
        x_baru = x2,
        error = error
      )
    )

    if (error < tol) {
      x1 <- x2
      break
    }

    x0 <- x1
    x1 <- x2
  }

  list(
    akar = x1,
    iterasi = hasil
  )
}
```

Jalankan:

```r
hasil_secant <- secant(
  f = f,
  x0 = 1,
  x1 = 2
)

hasil_secant$akar
```

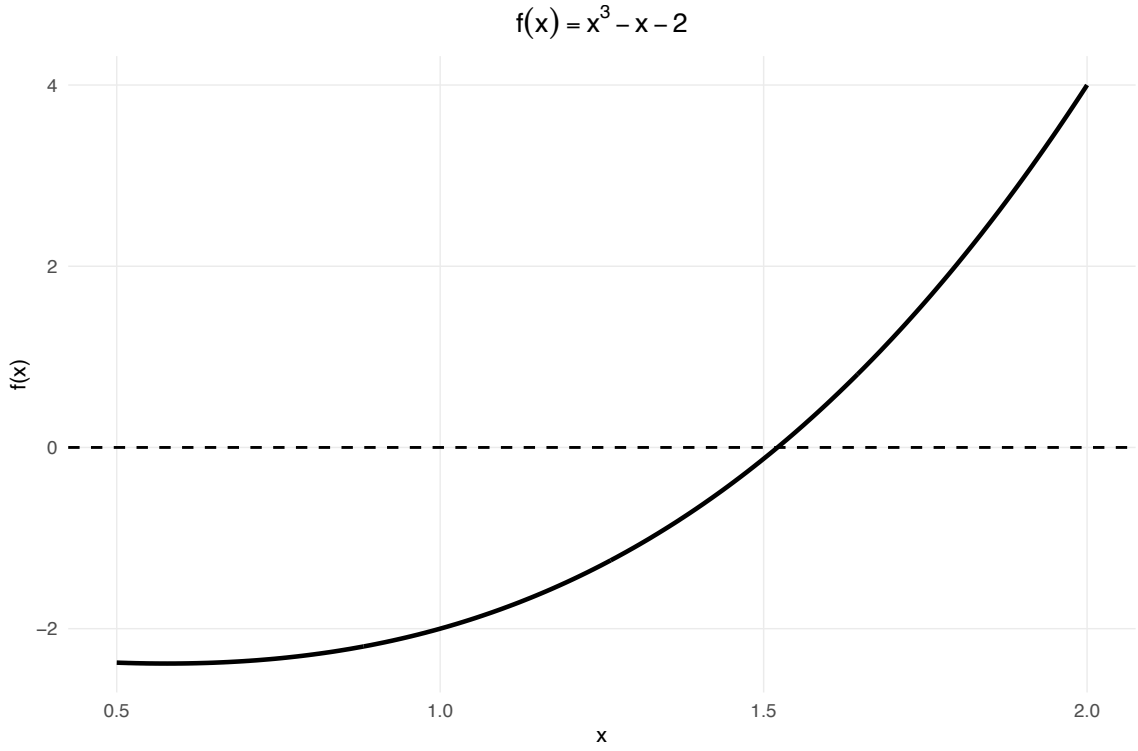
```
[1] 1.52138
```

```r
hasil_secant$iterasi
```

```
  iterasi x_sebelumnya x_sekarang   x_baru        error
1       1     1.000000   2.000000 1.333333 6.666667e-01
2       2     2.000000   1.333333 1.462687 1.293532e-01
3       3     1.333333   1.462687 1.531169 6.848286e-02
4       4     1.462687   1.531169 1.520926 1.024301e-02
5       5     1.531169   1.520926 1.521376 4.498962e-04
6       6     1.520926   1.521376 1.521380 3.391315e-06
7       7     1.521376   1.521380 1.521380 1.180307e-09
```

Visualisasi dan Perbandingan

Berdasarkan fungsi yang sama: $f(x) = x^3 - x - 2$, (lihat grafik)



Gambar 1: Grafik fungsi $f(x) = x^3 - x - 2$ dan lokasi akar persamaan.

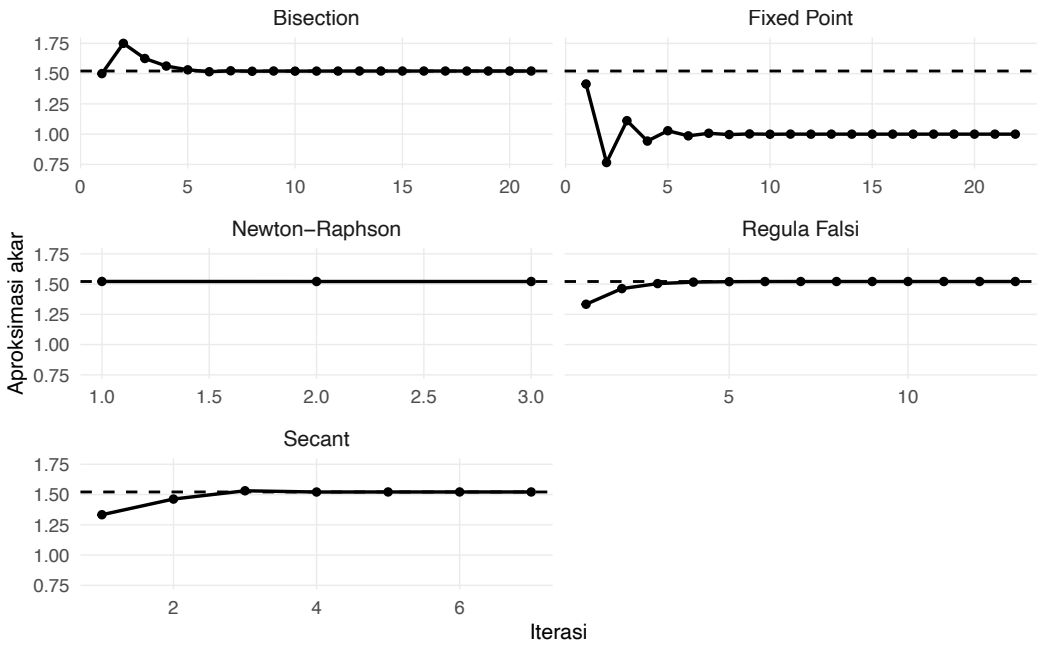
Berikut tabel perbandingannya

	Metode	Akar	Iterasi
1	Bisection	1.5213799	21
2	Regula Falsi	1.5213796	13
3	fixed point	0.9999998	22
4	Newton-Raphson	1.5213797	3
5	Secant	1.5213797	7

Plot berikut menunjukkan bagaimana nilai pendekatan akar berubah pada setiap iterasi untuk kelima metode.

Perhatikan hubungan antara grafik fungsi dan akar.

Pertanyaan:



Gambar 2: Konvergensi nilai akar dari kelima metode

1. Mengapa akar berada pada titik ketika Gambar 1 memotong sumbu-x?
2. Bagaimana interval awal dapat ditentukan dari Gambar 1?
3. Apa yang terjadi jika interval awal tidak mengandung perubahan tanda?
4. Apa yang bisa kalian simpulkan dari Gambar 2?

Praktikum 6 — Menerapkan Metode pada Berbagai Fungsi

Selanjutnya, beberapa fungsi non-linear berbeda akan digunakan dan dicoba untuk dicari akarnya. Tujuannya adalah menunjukkan bahwa metode pencarian akar dapat digunakan pada berbagai bentuk persamaan non-linear.

No.	Jenis Fungsi	Persamaan	Keterangan
1	Polinomial	$f_1(x) = x^2 - 5$	Akar positif: $x \approx 2.23607$
2	Polinomial Kubik	$f_2(x) = x^3 - 4x - 1$	Cari salah satu akar pada interval yang sesuai
3	Eksponensial	$f_3(x) = e^{-x} - x$	Akar: $x \approx 0.567143$
4	Trigonometri	$f_4(x) = \cos(x) - x$	Akar: $x \approx 0.739085$
5	Logaritmik	$f_5(x) = \ln(x) - 2$	Domain: $x > 0$
6	Fungsi Campuran	$f_6(x) = xe^{-x} - 0.2$	Cari akar pada interval yang sesuai

Catatan: Fungsi trigonometri dalam R menggunakan satuan **radian**.

Fungsi-fungsi ini memberikan contoh persamaan nonlinear yang tidak berbentuk polinomial sederhana.

Tugas 1 — Penerapan Terbimbing

Pilih tiga fungsi dari bagian sebelumnya. Untuk setiap fungsi:

1. Tentukan interval atau tebakan awal.
2. Jalankan metode *Bisection*.
3. Jalankan Regula Falsi.
4. Jalankan **fixed point** jika memungkinkan.
5. Jalankan Newton-Raphson.
6. Jalankan **Secant**.
7. Catat hasil akar.
8. Catat jumlah iterasi.
9. Catat *error*/galat.
10. Tentukan apakah metode konvergen.

Gunakan tabel berikut.

Fungsi	Metode	Tebakan/Interval	Akar	Iterasi	Error	Konvergen
f_1	Bisection					
f_1	Regula Falsi					
f_1	Newton					
f_1	*Secant*					

Fungsi	Metode	Tebakan/Interval	Akar	Iterasi	Error	Konvergen
...	...					

Tugas 2 — Mengubah Fungsi

Jangan mengubah algoritma metode yang sudah diberikan. Ubahlah hanya fungsi yang digunakan. Misalnya:

```
f_baru <- function(x) {
  cos(x) - x
}
```

Kemudian gunakan fungsi tersebut pada metode yang sesuai.

Contoh:

```
hasil <- bisection(
  f = f_baru,
  a = 0,
  b = 1
)

hasil$akar
```

```
[1] 0.7390852
```

Lakukan hal yang sama untuk fungsi nonlinear lainnya.

Tugas 3 — Eksperimen tebakan awal

Gunakan fungsi

$$f(x) = \cos(x) - x.$$

Gunakan Newton-Raphson dengan beberapa nilai awal:

$$x_0 = 0, \quad x_0 = 0.5, \quad x_0 = 1, \quad x_0 = 2.$$

Bandingkan hasilnya.

x_0	Akar	Iterasi	Konvergen?
0			
0.5			
1			
2			

Jawab:

1. Apakah semua tebakan awal menghasilkan akar yang sama?
2. Apakah jumlah iterasi sama?
3. Apa yang dapat disimpulkan mengenai pengaruh tebakan awal?

Tugas 4 — Eksperimen Nilai Toleransi

Gunakan salah satu metode dan fungsi

$$f(x) = x^3 - x - 2.$$

Gunakan beberapa nilai toleransi, misal:

```
10^-2
10^-4
10^-6
10^-8
```

Bandingkan:

- nilai akar,
- jumlah iterasi,
- error.

Buat tabel:

Toleransi	Akar	Iterasi	Error
10^{-2}			
10^{-4}			
10^{-6}			
10^{-8}			

Pertanyaan:

1. Apa yang terjadi ketika toleransi semakin kecil?
2. Apakah jumlah iterasi bertambah?
3. Apakah hasil akar menjadi lebih presisi?

Tugas 5 — Perbandingan Metode

Gunakan minimal tiga fungsi nonlinear yang berbeda. Bandingkan kelima metode berdasarkan:

1. jumlah iterasi,
2. nilai akar,
3. error,
4. kebutuhan turunan,
5. kebutuhan interval awal,
6. kestabilan,
7. kemudahan implementasi.

Lengkapi tabel:

Karakteristik	Bisection	Regula Falsi	*fixed point*	Newton-Raphson	*Secant*
Membutuhkan interval					
Membutuhkan turunan					
Membutuhkan dua tebakan awal					
Menggunakan $g(x)$					
Relatif stabil					

Karakteristik	Bisection	Regula Falsi	*fixed point*	Newton-Raphson	*Secant*
Cepat konvergen					

Tugas 6 — Analisis Konvergensi

Tidak semua metode selalu konvergen. Carilah contoh fungsi atau kondisi tebakan awal yang menyebabkan suatu metode **lambat, tidak konvergen, atau gagal dijalankan**. Jelaskan penyebabnya.

Metode	Hal yang perlu diperhatikan
Bisection	Apakah interval awal memiliki perubahan tanda?
Regula Falsi	Apakah interval awal memiliki perubahan tanda?
fixed point	Apakah $g(x)$ memenuhi kondisi konvergensi?
Newton-Raphson	Apakah $f'(x_n)$ mendekati nol?
Secant	Apakah $f(x_n) - f(x_{n-1})$ mendekati nol?

Refleksi

Jawab pertanyaan berikut dengan kalimat sendiri.

1. Apa perbedaan ide dasar kelima metode?
2. Mengapa *Bisection* disebut sebagai metode yang menjamin penyempitan interval?
3. Apa perbedaan *Bisection* dan *Regula Falsi*?
4. Mengapa Newton-Raphson dapat lebih cepat daripada *Bisection*?
5. Apa kelemahan Newton-Raphson?
6. Mengapa *Secant* tidak membutuhkan turunan?
7. Mengapa *fixed point* membutuhkan pemilihan $g(x)$ yang tepat?
8. Apakah metode yang paling cepat selalu merupakan metode yang paling baik?
9. Apa peran R dalam penyelesaian masalah numerik?

Tugas Akhir Praktikum

Pilih **satu persamaan nonlinear** yang berbeda dari contoh utama, misalnya polinomial, eksponensial, logaritmik, trigonometri, atau kombinasi fungsi. Gunakan minimal **tiga metode** pencarian akar.

Laporan memuat:

1. **Persamaan dan grafik** — Tuliskan fungsi dan tampilkan grafik untuk memperkirakan lokasi akar.
2. **Perhitungan manual** — Tunjukkan minimal 3 iterasi untuk setiap metode.
3. **Algoritma** — Jelaskan langkah metode dengan bahasa sendiri.
4. **Implementasi R** — Tampilkan kode yang digunakan.
5. **Hasil** — Sajikan akar, jumlah iterasi, *error*, dan kriteria berhenti.
6. **Perbandingan dan pembahasan** — Bandingkan kecepatan, kestabilan, pengaruh tebakan awal/interval, serta kemungkinan kegagalan metode.
7. **Kesimpulan** — Tuliskan kesimpulan berdasarkan hasil eksperimen.

Ringkasan Lima Metode

Metode	Ide Utama	Informasi yang Dibutuhkan
Bisection	Mempersempit interval	a, b
Regula Falsi	Menggunakan garis lurus	a, b
fixed point	Menghasilkan x baru dari $g(x)$	$x_0, g(x)$
Newton-Raphson	Menggunakan garis singgung	$x_0, f'(x)$
Secant	Menggunakan garis melalui dua titik	x_0, x_1

Secara konseptual:

$$\boxed{\text{Persamaan} \rightarrow \text{Perhitungan Manual} \rightarrow \text{Algoritma} \rightarrow \text{R} \rightarrow \text{Eksperimen} \rightarrow \text{Analisis}}$$

Inilah alur utama praktikum metode numerik.